

Tecnico Superiore per lo sviluppo di sistemi full stack per web, mobile e desktop
ITS Software Architect Specialist

Biennio 2023-25

Improving without downtime:
Normalizzazione di un Database in uso

Luca Ilari

Disclaimer	3
Chi sono	3
Widesolutions.it	4
Il mio ruolo	4
Corsi di formazione seguiti	5
Attività svolte durante lo stage	5
Tecnologie e strumenti utilizzati	6
Analisi database e architettura attuale della web app	7
Architettura della web app	8
Integrazione con piattaforme esterne	9
Sviluppo di report	10
Analisi e risoluzione bug su processi esistenti	13
Caso di bug: concorrenza tra activation bot e housekeeping	14
Un approfondimento sul WITH (NOLOCK), pro e contro	16
Caso di bug: differenza di tipo tra colonne correlate	17
Cos'è una collation?	18
Analisi e risoluzione	18
Analisi e normalizzazione dell'architettura dati	18
Analisi architettura attuale	19
Problematiche di un database non correttamente normalizzato	19
Esempio pratico di duplicazione	20
Nuovo modello proposto	21
Rimozione del collegamento con idViaggio	21
Gestione della disponibilità con flag dedicati	21
Esempio pratico	21
Impatto e rischi delle modifiche sui processi attuali	22
Impatto sulle stored procedure e sulla logica esistente	22
Rischi principali	23
Soluzioni applicate	23
Strategia di migrazione senza interruzioni	23
Conclusioni	25
Bibliografia/Sitografia	26

Disclaimer

Durante il mio stage presso Widesolutions.it, ho svolto prevalentemente attività di consulenza per un'azienda cliente estera.

Per motivi di riservatezza e in rispetto degli accordi di non divulgazione, non posso divulgare il nome dell'azienda né fornire dettagli specifici che possano identificarla.

Per proteggere la privacy e la riservatezza delle informazioni aziendali, nella tesi sono presenti screenshot, dati e documenti opportunamente censurati e anonimizzati. Gli esempi utilizzati sono stati creati appositamente e non rappresentano dati reali, ma servono a illustrare i concetti e le metodologie sviluppate durante il progetto.

In particolare, ho realizzato esempi della struttura del database che, insieme agli screenshot e alle mappe architetture, aiutano a comprendere la struttura generale dell'applicazione su cui ho lavorato e a evidenziare i problemi che, insieme al mio team, ho dovuto affrontare e risolvere durante lo stage.

Chi sono

Mi chiamo Luca Ilari, sono nato a Milano e ora abito a Mediglia, un piccolo paese in periferia. Mi sono appassionato all'informatica alle scuole medie, quando cercavo di capire come funzionasse un computer e come riuscisse a farmi vedere dei pixel colorati sullo schermo. Mi sembrava incredibile che da "qualcosa di invisibile" potessero uscire immagini, giochi e video... per me era davvero una specie di magia. Sin da subito ho provato una forte passione nel creare piccoli programmi e giochi, utilizzando software come Scratch, Processing e Arduino, che mi hanno aiutato a comprendere meglio il funzionamento di un PC e le tecniche base della programmazione.

Una volta terminate le medie, ho deciso di intraprendere la strada dell'istituto tecnico informatico perché amavo l'idea di fare qualcosa di pratico che mi potesse insegnare concretamente lo sviluppo software. Sin dal primo anno abbiamo realizzato progetti di programmazione molto semplici, che mi hanno permesso di apprendere le basi dello sviluppo software. Visto che l'interesse era reale, ho iniziato a dedicarmi a molti progetti personali per migliorare le mie conoscenze, partendo dalle basi come la simulazione dei gate logici utilizzando Logisim, fino ad arrivare alla creazione di siti web. Questi esperimenti mi hanno permesso di consolidare le competenze teoriche apprese a scuola e di sperimentare con tecnologie diverse, come i database e linguaggi di programmazione nuovi per me.

Una volta terminate le superiori, ho deciso di iscrivermi all'ITS Angelo Rizzoli, che mi ha aiutato enormemente a collegare tutte le informazioni e le competenze che avevo acquisito in un percorso pratico e orientato al lavoro. Grazie a questo percorso formativo ho potuto affrontare progetti concreti e collaborare con aziende del settore, trasformando così la mia passione in una vera e propria professione.



è una azienda di consulenza informatica con una ventina di dipendenti circa e con molti anni di esperienza. Si occupa prevalentemente di consulenza informatica con vari clienti in diversi settori come viaggi, turismo e bancario.

Lo stage è stato fatto prevalentemente in smart working, essendo che il cliente per cui ho lavorato ha sede al di fuori dell'Italia.

Il lavoro da remoto è un aspetto fondamentale di Widesolutions che essendo formata da consulenti sparsi in tutta Italia non richiede di lavorare presso un ufficio; questo permette di avere una vita più equilibrata facendo risparmiare il tempo dei viaggi dall'ufficio a casa e viceversa. Allo stesso tempo, non mancano occasioni per incontrarsi di persona e mantenere un contatto umano. Infatti l'azienda organizza periodicamente delle giornate chiamate "Beautiful Days", in cui tutti i dipendenti si ritrovano in spazi di coworking per passare del tempo insieme di persona. L'idea è quella di staccare un po' dalla routine lavorativa, rilassarsi, e allo stesso tempo partecipare ad attività che aiutano a crescere personalmente e professionalmente.

Durante queste giornate ci sono presentazioni, momenti di condivisione e occasioni per parlare apertamente, anche per dare feedback su eventuali problemi o suggerire miglioramenti. È un modo per confrontarsi in modo costruttivo e sentirsi parte attiva dell'azienda, in un clima sereno e collaborativo.

Oltre ai "Beautiful Days", durante il mio stage il cliente con cui ho lavorato ha chiesto più volte di organizzare incontri dal vivo con il team. Questi momenti si sono rivelati molto utili, soprattutto per affrontare questioni urgenti o per fare il punto della situazione in modo più diretto ed efficace. Il confronto in presenza ha aiutato molto a chiarire dubbi, prendere decisioni più velocemente e migliorare il lavoro di squadra.

Il mio ruolo

Nel corso dello stage, il mio ruolo è cresciuto progressivamente: da semplice supporto al team di sviluppo sono arrivato, con il tempo, a essere una figura di riferimento su alcune attività specifiche. All'inizio affiancavo i colleghi più esperti per comprendere le dinamiche del progetto e acquisire familiarità con il contesto tecnico e funzionale. Dopo una prima fase di apprendimento, ho iniziato a essere un punto d'appoggio per il team, soprattutto su task ripetitivi o più analitici, permettendo così agli altri membri di concentrarsi su aspetti più complessi.

Col tempo ho guadagnato sempre più autonomia, prendendomi carico di analisi più approfondite e sviluppi che richiedevano una buona comprensione sia della logica applicativa sia delle esigenze del cliente. In diverse occasioni, sono stato io a proporre soluzioni tecniche o a chiarire alcuni dubbi del team grazie alla mia conoscenza sempre più solida del sistema e della base dati. Questo passaggio da “chi impara” a “chi supporta” è stato uno degli aspetti più gratificanti della mia esperienza.

A fine stage, ero in grado di gestire interamente alcune attività, dall'analisi delle richieste fino allo sviluppo vero e proprio, con una buona indipendenza. Ho imparato a organizzare il mio lavoro, a capire le priorità e a confrontarmi in modo più diretto con i referenti tecnici del progetto. Questo mi ha permesso non solo di crescere professionalmente, ma anche di sentirmi parte integrante del team, contribuendo attivamente al successo del progetto.

Corsi di formazione seguiti

Durante lo stage presso Widesolutions ho avuto l'opportunità di seguire diversi corsi di formazione, che si sono rivelati molto utili nello svolgimento delle attività quotidiane.

Il primo corso seguito riguardava il T-SQL (Transact-SQL), un'estensione del linguaggio SQL sviluppata da Microsoft, che introduce funzionalità aggiuntive per la creazione di stored procedures e script come per esempio l'utilizzo di tabelle temporanee, variabili, cicli, condizioni e molte altre funzionalità avanzate. La conoscenza del T-SQL si è rivelata fondamentale per il lavoro svolto durante lo stage, poiché il database utilizzato dal cliente era basato su Microsoft SQL Server.

Oltre al corso su T-SQL, ho seguito anche altri corsi di formazione su diversi argomenti, tra cui Jenkins, Ansible, intelligenza artificiale e sicurezza informatica. Tutti questi percorsi si sono dimostrati molto interessanti e mi hanno permesso di ampliare la mia visione sugli strumenti di programmazione, anche se non li ho utilizzati direttamente nelle attività quotidiane.

Attività svolte durante lo stage

Il mio stage presso Widesolutions è iniziato il 13 gennaio e, appena completato il corso su T-SQL sono stato inserito nel team che segue un'importante multinazionale nel settore delle crociere di lusso. Per motivi di riservatezza, nei prossimi paragrafi mi riferirò a questa azienda semplicemente come “il cliente”.

Sono stato aggiunto a questo progetto perché era un ambito che rispecchia le mie competenze e i miei interessi ovvero la programmazione backend e la gestione dei database.

Fin da subito ho potuto partecipare attivamente alle varie attività del team, lavorando a fianco dei colleghi e imparando direttamente partecipando ai loro incarichi giornalieri.

Il mio team si occupa della manutenzione e dello sviluppo di una sezione cruciale del database SQL Server che supporta una web app usata quotidianamente dal cliente e dai tour operator affiliati. In particolare io e il mio team abbiamo lavorato alla parte che gestisce la creazione, modifica e prenotazione di tutta una serie di servizi sulle navi e fuori; come per esempio escursioni fuori bordo, spa e prenotazioni nei ristoranti della nave.

Durante il periodo di stage ho svolto diverse attività, spaziando da compiti di natura analitica a quelli più tecnici. In particolare, mi sono occupato della creazione di report personalizzati sulla base delle richieste del team commerciale del cliente, che necessitava di analisi specifiche, ad esempio sui dati delle prenotazioni relative a determinati tour terrestri.

Parallelamente, ho contribuito allo sviluppo di nuove funzionalità per l'applicazione su cui stavo lavorando e alla progettazione di modifiche all'architettura del database, con l'obiettivo di migliorare le performance e supportare l'introduzione di nuove esigenze funzionali richieste dal cliente.

Tecnologie e strumenti utilizzati

Durante lo stage ho avuto modo di lavorare con diversi strumenti. Alcuni li conoscevo già ma utilizzarli in un contesto lavorativo reale mi ha permesso di approfondire molto di più il loro funzionamento e capire come vengono impiegati nella quotidianità.

Uno degli strumenti che ho utilizzato più spesso è stato SQL Server Management Studio (SSMS), che mi è servito per interagire con il database della piattaforma su cui ho lavorato. Con SSMS ho potuto scrivere e testare query SQL, eseguire operazioni di manutenzione sui dati e monitorare il comportamento delle Stored procedures già esistenti. È stato fondamentale per comprendere la struttura del database e per intervenire in modo mirato quando serviva modificare o ottimizzare qualcosa.

Ho anche imparato a utilizzare SQL Server Profiler, uno strumento a me nuovo fino a quel momento. Mi è stato molto utile soprattutto per fare debug e analizzare il funzionamento della webapp e della sua interazione con il database, perché mi permetteva di tracciare quali Stored Procedure venivano chiamate e in quale sequenza, durante l'esecuzione delle operazioni. Questo mi ha aiutato a individuare rapidamente i punti critici e a risolvere bug in diversi progetti, migliorando così la mia comprensione del flusso di lavoro interno al database.

Per quanto riguarda lo sviluppo vero e proprio, ho utilizzato Visual Studio con .NET 6, lavorando su progetti backend scritti in C#. Mi sono occupato sia della scrittura di nuove funzionalità, sia della modifica di codice esistente, cercando di seguire le linee guida del team in termini di struttura e gestione degli errori.

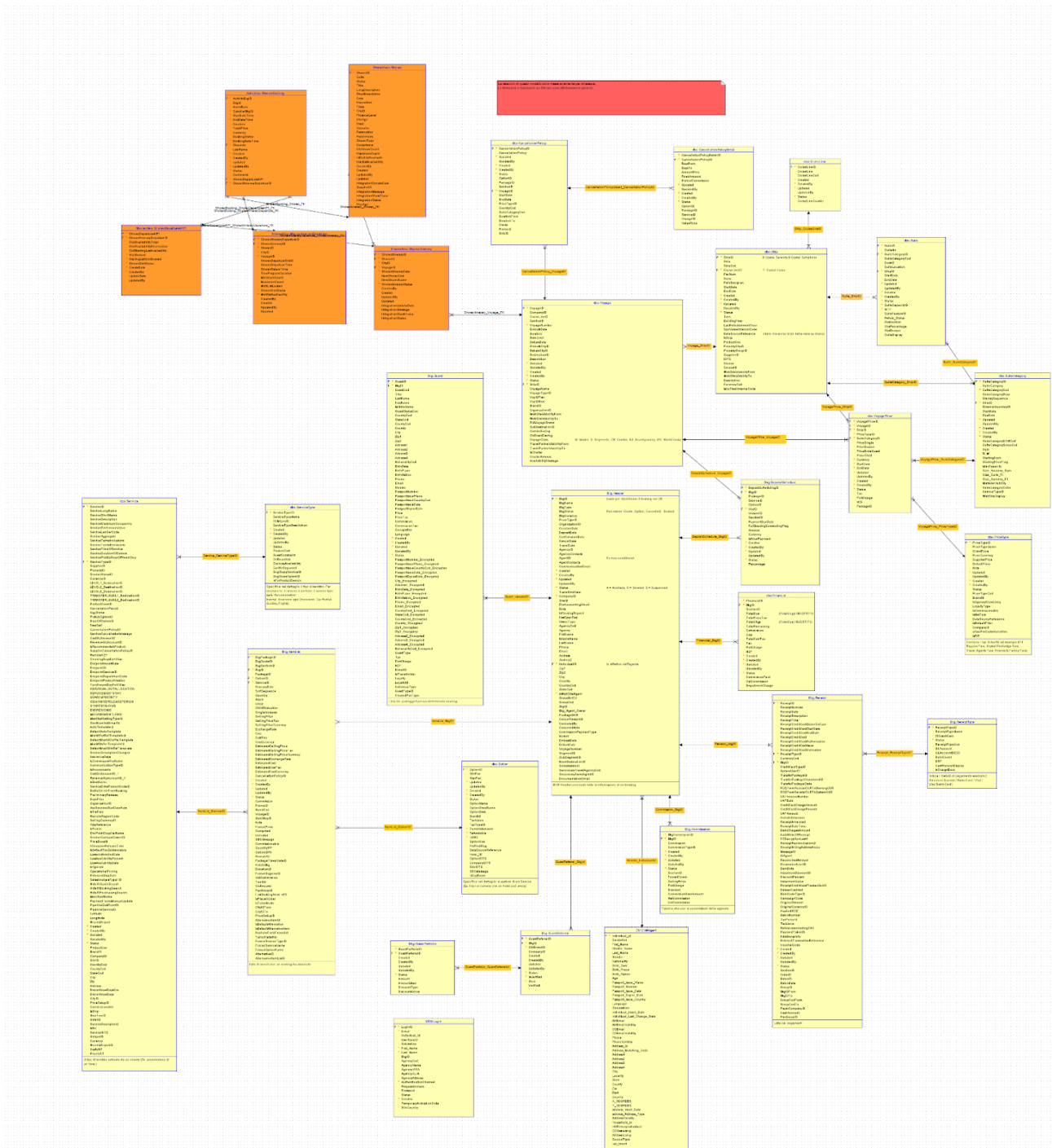
Un altro strumento che ho imparato ad apprezzare è stato Redgate, in particolare per la gestione del versionamento del database e per confrontare facilmente gli script SQL tra ambienti diversi (ad esempio sviluppo e produzione). Redgate ha velocizzato molto alcune attività che altrimenti sarebbero risultate più macchinose, come il deploy di modifiche o il controllo delle differenze tra versioni.

In generale, questi strumenti mi hanno permesso di entrare nel flusso di lavoro dell'azienda e di contribuire concretamente ai progetti in corso. Ogni strumento aveva la sua utilità specifica, ma è stato il loro utilizzo combinato che mi ha fatto davvero capire quanto sia importante avere un buon ecosistema tecnico a supporto del lavoro quotidiano.

Analisi database e architettura attuale della web app

Uno dei primi compiti che mi è stato assegnato è stato quello di creare un diagramma relazionale con le tabelle più comunemente usate dal team per farmi prendere familiarità con la struttura del database. Questa task ha subito migliorato la mia comprensione della struttura generale del sistema e mi ha permesso anche di approfondire i vincoli architetturali che regolano le relazioni tra i dati, come chiavi primarie, chiavi esterne e dipendenze logiche. Lavorare alla creazione del diagramma relazionale è stato inoltre utile per tutto il team, poiché fino a quel momento non esisteva un documento visuale aggiornato e condiviso che rappresentasse chiaramente l'interconnessione tra le principali entità del database. Il diagramma ha quindi colmato una lacuna importante, diventando un riferimento pratico e immediato per comprendere e navigare più facilmente nella base dati, soprattutto per i nuovi membri del team e durante le fasi di sviluppo e debugging.

Di seguito è riportato uno screenshot esemplificativo del diagramma relazionale, inserito con l'unico scopo di dare un'idea generale della complessità e dell'organizzazione della base dati. Per motivi di riservatezza, nello schema non sono visibili i nomi delle tabelle né i dettagli delle relazioni: si tratta di una vista parziale e semplificata, utile solo a livello illustrativo. Le tabelle rappresentate nello screenshot sono solo una parte delle entità effettivamente utilizzate nel sistema, mentre altre sezioni più specifiche del database verranno descritte e analizzate in modo più approfondito nei capitoli successivi.



Architettura della web app

Successivamente all'analisi del database, ho avuto modo di approfondire anche l'architettura generale della web app, per comprendere meglio in che modo i vari componenti del sistema interagiscono tra loro. L'applicazione è strutturata secondo un'architettura a più livelli, in cui ogni parte svolge un ruolo specifico e ben definito.

Il database è un Microsoft SQL server e risiede su un server separato, accessibile unicamente tramite una connessione VPN, che garantisce un livello adeguato di sicurezza, soprattutto nei contesti di accesso da remoto. A questo livello si trova anche una parte sostanziale della business logic dell'applicazione, implementata direttamente attraverso stored procedure (SP). Queste procedure vengono richiamate dal backend per eseguire le principali operazioni sui dati, come letture complesse, scritture e controlli specifici. Questa scelta architetturale ha permesso al team di centralizzare la logica applicativa più critica direttamente nel database, migliorando la coerenza dell'elaborazione dei dati e riducendo la duplicazione del codice tra backend e database.

Il backend è sviluppato in .NET ed ospitato su un server Azure. Il suo compito principale è quello di fungere da intermediario tra frontend e database, infatti riceve le richieste dal client, valida i parametri, esegue le chiamate alle stored procedure e restituisce i risultati in formato JSON, esponendo una serie di API REST. Il backend include inoltre meccanismi di logging, gestione degli errori e controllo degli accessi, fondamentali per garantire l'affidabilità e la sicurezza del sistema.

Il frontend dell'applicazione è una web app sviluppata con Next.js, un framework basato su React che permette di combinare rendering lato server e rendering lato client, migliorando così la performance generale e l'ottimizzazione SEO. L'interfaccia è progettata per essere moderna e reattiva, con componenti modulari facilmente riutilizzabili. L'interazione con il backend avviene tramite chiamate HTTP sicure (HTTPS), attraverso le quali il frontend accede alle API esposte e ottiene i dati necessari per la visualizzazione e l'interazione con l'utente. Anche questo collegamento tra web server e backend avviene dentro una VPN per evitare di esporre gli endpoint REST al di fuori dell'applicazione.

Nel complesso, l'architettura risulta solida e ben strutturata, con una chiara separazione delle responsabilità tra i vari livelli. L'adozione di stored procedure per la logica di business ha semplificato alcune dinamiche di sviluppo e manutenzione, permettendo una gestione più centralizzata e coerente delle regole applicative legate ai dati.

Integrazione con piattaforme esterne

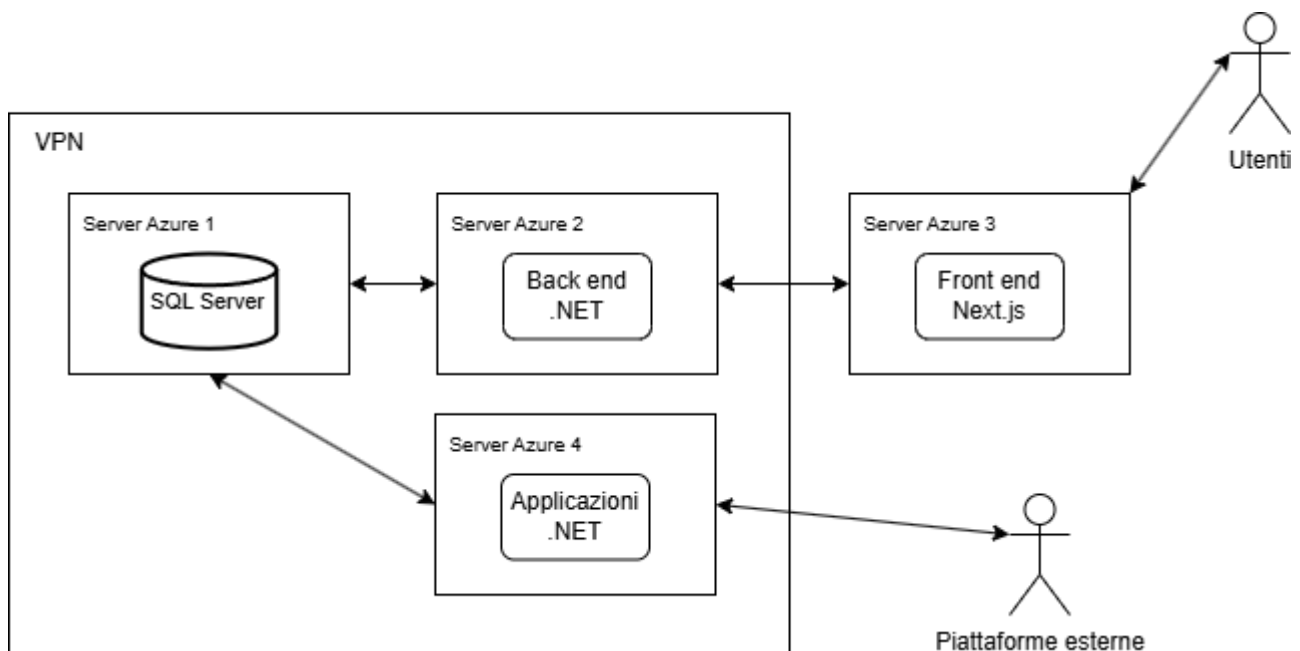
Un ulteriore elemento dell'architettura che ho avuto modo di approfondire riguarda il sistema di integrazione con piattaforme esterne, necessario per garantire che i dati presenti nella web app siano condivisi e sincronizzati con altri ambienti o strumenti utilizzati all'interno dell'organizzazione o da soggetti terzi.

Per gestire questi flussi, è stato predisposto un server separato, incaricato di estrarre i dati dal database centrale, elaborarli secondo le regole previste e inviarli verso delle piattaforme esterne. Questo server opera in maniera autonoma rispetto al backend principale, con il compito specifico di assicurare lo scambio costante e affidabile delle informazioni tra sistemi differenti.

L'estrazione dei dati avviene principalmente tramite stored procedure dedicate, che restituiscono insiemi di dati già filtrati e formattati in base alle esigenze dell'integrazione del sistema esterno. Una volta estratti, i dati vengono ulteriormente trasformati, se necessario, e inviati alle destinazioni previste. Il sistema include controlli di validità, gestione degli errori e logging, in modo da garantire tracciabilità e affidabilità in tutte le fasi del processo.

L'utilizzo di un server dedicato per queste attività consente di isolare il carico legato all'integrazione, evitando interferenze con il funzionamento ordinario della web app e del backend. Questo approccio favorisce una maggiore scalabilità e una manutenzione più semplice, oltre a rendere più flessibile l'adattamento futuro verso nuove piattaforme o modalità di scambio dati.

Di seguito un diagramma approssimato con riportata l'architettura appena illustrata



Sviluppo di report

Lo sviluppo di report ha rappresentato una parte significativa del mio stage. I report che ho realizzato possono essere suddivisi in due principali categorie:

1. Report richiesti dal team commerciale
2. Report utilizzati per identificare discrepanze e bug nei dati presenti nel database

I report richiesti dal team commerciale erano principalmente finalizzati all'analisi della disponibilità dei tour e alla valutazione del gradimento dei passeggeri. Questi report risultano generalmente più rapidi da sviluppare, poiché richiedono la creazione di query specifiche che non comportano l'incrocio di molte tabelle.

Tutti i report vengono salvati all'interno di una sezione dedicata della web app, chiamata MyReport. Questa sezione consente di esporre i risultati delle Stored Procedure create appositamente per ogni tipo di analisi.

Di seguito è riportato un esempio di report da me sviluppato e integrato nella sezione MyReport. In questo caso, il report è stato progettato per individuare i tour terrestri che non avevano un prezzo assegnato oppure presentavano una validità di prezzo scaduta. L'obiettivo era quindi facilitare l'identificazione di questi casi, in modo da poter procedere con l'inserimento di un nuovo prezzo.

Shorex - no Price

TITLE	CITY NAME	COUNTRY NAME	PRICE VALIDITY	LAST PRICE ADULT	LAST PRICE CHILD	LAST PRICE VEHICLE	IS A FUTURE CALL	STATUS
	Fremantle (P...	Australia	Expired on: 2024-12-31	119.00			No	Final Approved
	Abu Dhabi	United Arab Emirates	Expired on: 2024-12-31			419.00	Yes	Final Approved
	Abu Dhabi	United Arab Emirates	Expired on: 2024-12-31			809.00	Yes	Final Approved
	Abu Dhabi	United Arab Emirates	Expired on: 2024-12-31			529.00	Yes	Final Approved
	Abu Dhabi	United Arab Emirates	Expired on: 2024-12-31			939.00	Yes	Final Approved
	Abu Dhabi	United Arab Emirates	Expired on: 2024-12-31	119.00			Yes	Final Approved
	Abu Dhabi	United Arab Emirates	Expired on: 2024-12-31	109.00			Yes	Final Approved
	Abu Dhabi	United Arab Emirates	Expired on: 2024-12-31	219.00			Yes	Final Approved
	Abu Dhabi	United Arab Emirates	Expired on: 2024-12-31	529.00			Yes	Final Approved
	St. Kitts (Bas...	St Kitts and Nevis	Expired on: 2025-05-31	99.00	69.00		Yes	Final Approved
	St. Kitts (Bas...	St Kitts and Nevis	Expired on: 2025-05-31	79.00			Yes	Final Approved
	St. Kitts (Bas...	St Kitts and Nevis	Expired on: 2025-05-31	119.00			Yes	Final Approved
	St. Kitts (Bas...	St Kitts and Nevis	Expired on: 2025-05-31	99.00	49.00		Yes	Final Approved

Page 1 of 13 (1259 items)

12345...13

I report sviluppati per identificare bug e discrepanze dei dati sono stati più impegnativi, in quanto ho dovuto analizzare in profondità la struttura del database, individuare eventuali anomalie nei dati e progettare query complesse in grado di evidenziare situazioni non coerenti o potenzialmente errate. Questo ha richiesto non solo una buona conoscenza del modello relazionale e delle logiche applicative del sistema, ma anche la capacità di interpretare correttamente il significato funzionale dei dati. Questi report hanno avuto un ruolo fondamentale nel processo di miglioramento della qualità del dato, poiché hanno permesso di prevenire errori a valle e di intervenire tempestivamente sulle cause alla radice.

Un esempio concreto riguarda la realizzazione di un report volto a individuare viaggi con itinerari differenti ma che, in realtà, avrebbero dovuto essere identici. Questo tipo di errore

può verificarsi a causa della scarsa normalizzazione della tabella degli itinerari, dove la stessa nave può avere più viaggi programmati per lo stesso giorno, ma con itinerari non allineati, spesso a causa di differenze minime o duplicazioni involontarie.

Ecco un esempio per capire meglio il problema con la tabella degli itinerari semplificata e con dati di esempio:

IdViaggio	IdNave	DataItinerario	Posizione	Porto
1001	2	2025-08-10	1	Savona
1001	2	2025-08-10	2	Genova
1001	2	2025-08-10	<u>3</u>	<u>La Spezia</u>
1002	2	2025-08-10	1	Savona
1002	2	2025-08-10	2	Genova
1002	2	2025-08-10	<u>3</u>	<u>Portovenere</u>

In questo esempio, i viaggi con **IdViaggio 1001** e **1002** sono associati alla stessa nave (**IdNave 2**) e alla stessa data itinerario (**2025-08-10**), ma presentano itinerari differenti, ovvero il porto in **Posizione 3** varia da **La Spezia** a **Portovenere**. In molti casi, questa situazione rappresenta un errore, poiché in fase di caricamento manuale o duplicazione, le modifiche non sono state propagate correttamente tra i viaggi che avrebbero dovuto condividere lo stesso percorso.

Per individuare queste discrepanze, ho progettato una logica che analizza i viaggi effettuati dalla stessa nave nello stesso giorno, verificando se condividono effettivamente lo stesso itinerario. Per fare questo, ho usato la funzione ROW_NUMBER() per assegnare a ogni tappa una posizione progressiva (DayPos) all'interno del viaggio, ordinando le tappe secondo il campo Posizione. Questo permette di confrontare facilmente, tra viaggi diversi, la sequenza dei porti giorno per giorno.

Successivamente, vengono confrontati tutti i viaggi con stesso IdNave e DataItinerario. Se per una stessa DayPos (es. la terza tappa del viaggio) due viaggi presentano porti diversi, il report li segnala come potenzialmente errati. In questo modo è possibile individuare rapidamente incoerenze dovute a duplicazioni non corrette, modifiche parziali o inserimenti sbagliati.

Ecco una pseudo query semplificata per rappresentare questa logica:

```
-- Assegna una posizione giornaliera ordinata per ogni viaggio
SELECT
    IdViaggio,
    IdNave,
    DataItinerario,
    Porto,
    ROW_NUMBER() OVER (PARTITION BY IdViaggio, DataItinerario ORDER BY Posizione) AS DayPos
FROM Itinerari
INTO #ItinerariRanked --Inserimento in una tabella temporanea
WHERE Stato = 'Attivo';

-- Confronta viaggi della stessa nave nello stesso giorno
SELECT
    v1.IdViaggio AS Viaggio1,
    v2.IdViaggio AS Viaggio2,
    v1.DataItinerario,
    v1.DayPos,
    v1.Porto AS Porto1,
    v2.Porto AS Porto2
FROM #ItinerariRanked v1
JOIN #ItinerariRanked v2
    ON v1.IdNave = v2.IdNave
    AND v1.DataItinerario = v2.DataItinerario
    AND v1.DayPos = v2.DayPos
WHERE v1.Porto <> v2.Porto; --Verifica discrepanza porti
```

Questo approccio ha permesso di identificare numerosi casi di disallineamento tra viaggi simili, migliorando la coerenza dei dati e prevenendo errori a valle nei processi di prenotazione e pubblicazione itinerari.

Analisi e risoluzione bug su processi esistenti

Durante lo stage, una parte rilevante del lavoro ha riguardato l'analisi e la risoluzione di bug riscontrati nei processi già in uso all'interno del sistema. Questo tipo di attività si è rivelato particolarmente formativo, poiché mi ha permesso di approfondire il modo in cui affrontare i problemi e di come andare ad analizzare la causa effettiva di un bug. Inoltre, mi ha dato l'opportunità di confrontarmi con i colleghi e di migliorare il mio approccio al lavoro in team.

Prima di portare un esempio concreto di un bug risolto, è utile spiegare due processi importanti su cui ho lavorato direttamente: l'activation bot e il processo di housekeeping.

L'**activation bot** è uno script SQL utilizzato per generare in modo automatico tutte le attività terrestri (tuor) previste per le navi per un determinato anno. È uno strumento fondamentale, poiché consente la creazione automatica e veloce di dati che una persona impiegherebbe giorni ad inserire manualmente. In pratica, ogni volta che c'è bisogno di inserire dei tour prenotabili per un nuovo anno, viene eseguito questo script, che genera una serie di attività

prenotabili dai clienti per tutto il periodo scelto. Queste attività vengono create incrociando i dati degli itinerari delle navi, gli orari di arrivo e di partenza e molti altri dati.

Il **job di housekeeping**, invece, è un processo schedato che viene eseguito ogni 5 minuti e ha l'obiettivo di ripulire, correggere e sincronizzare i dati inseriti nelle tabelle dei tour. Questo è necessario perché alcune tabelle del database non sono normalizzate e non sono collegate da vincoli con chiavi primarie e chiavi esterne, il che rende possibile l'inserimento di record non coerenti o incompleti. Il compito dell'housekeeping è quindi quello di monitorare costantemente queste situazioni e di correggerle in automatico, rigenerando i dati mancanti o errati.

Caso di bug: concorrenza tra activation bot e housekeeping

Un caso particolare è stato quello legato a un comportamento inatteso causato dall'interazione tra i due processi descritti nel paragrafo prima, ovvero, l'activation bot e il task di housekeeping.

Nello specifico, stavamo facendo dei test sullo script di attivazione dei tour per un determinato anno, poiché avevo apportato alcune modifiche alle logiche di creazione delle attività. Dopo aver verificato il corretto funzionamento nell'ambiente di UAT, sono passato a effettuare un test anche in produzione, ma in modo controllato: ho eseguito lo script all'interno di un blocco BEGIN TRANSACTION / ROLLBACK per assicurarmi che nessun dato venisse effettivamente scritto nel database, evitando così di inserire informazioni non corrette nell'ambiente live.

Lo script, infatti, esegue tutte le sue operazioni in una transazione e finché non viene eseguito un COMMIT, tutte le modifiche restano visibili solo all'interno della sessione corrente e non sono salvate in modo permanente. Di seguito è riportato un estratto dello script per mostrare come è strutturato.

```
BEGIN TRANSACTION
```

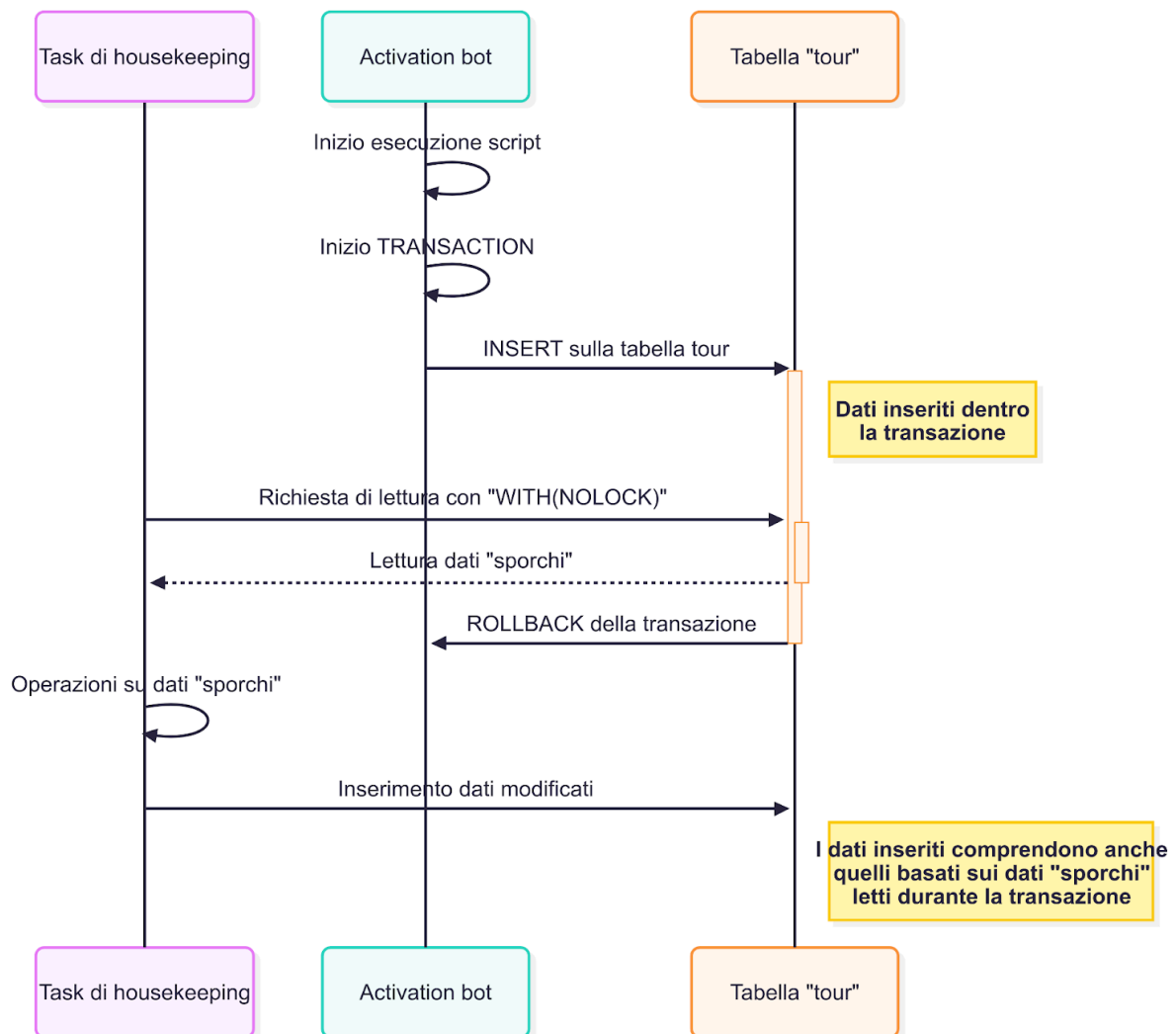
```
-- Processi di estrazione ed inserimento dei  
-- nuovi dati relativi ai tour da creare
```

```
ROLLBACK TRANSACTION
```

Tuttavia, in questa occasione, proprio mentre lo script era in esecuzione e la transazione era ancora aperta, è partito anche il processo di housekeeping. Questo ha generato un effetto collaterale imprevisto, perché l'housekeeping ha letto anche i dati temporanei creati dall'activation bot, causando un comportamento errato che ha richiesto un intervento correttivo.

Per capire meglio cosa è successo e come si è generato il bug, è utile vedere nel dettaglio l'ordine esatto delle operazioni e come i due processi (activation bot e housekeeping) sono entrati in conflitto.

Il diagramma seguente rappresenta visivamente il flusso degli eventi e aiuta a chiarire in che modo il processo di housekeeping sia riuscito a leggere dei dati ancora "temporanei" e non confermati, causando così la creazione di informazioni duplicate.



Il diagramma sopra mostra passo dopo passo l'interazione tra l'activation bot, il task di housekeeping e la tabella dei tour, evidenziando il flusso che ha portato alla creazione di dati inconsistenti.

Inizio esecuzione dell'activation bot: lo script SQL viene eseguito manualmente e inizia con un blocco BEGIN TRANSACTION, che racchiude tutte le operazioni successive per assicurarne l'atomicità.

Inserimento dati: lo script effettua una serie di INSERT nella tabella dei tour. Tuttavia, essendo ancora all'interno della transazione, questi dati non sono visibili globalmente nel database.

Intervento dell'housekeeping: proprio in quel momento, parte in automatico il processo di housekeeping. Una delle sue query effettua una SELECT dalla tabella dei tour utilizzando WITH (NOLOCK), che permette di leggere i dati anche se non sono stati ancora confermati con il comando (COMMIT TRANSACTION).

Di seguito una query simile a quella utilizzata dentro il processo di Housekeeping

```
--Task di housekeeping
SELECT
    id,
    title,
    ...
FROM tours WITH(NOLOCK)
```

Lettura dei dati "sporchi": la query dell'housekeeping riesce a vedere i dati temporanei inseriti dalla transazione ancora aperta dell'activation bot. Questi dati, benché non ancora ufficiali, vengono considerati validi.

Rollback della transazione: il test finisce e viene eseguito un ROLLBACK, annullando tutte le modifiche effettuate dall'activation bot. A questo punto, i dati letti dall'housekeeping non esistono più nel database.

Elaborazione errata dell'housekeeping: il task continua a elaborare quei dati "sporchi" come se fossero corretti e crea nuovi record basandosi su quelle informazioni, portando alla creazione di duplicati e dati inconsistenti.

In pratica, il problema nasce proprio dall'uso della clausola WITH (NOLOCK), che permette di accedere a dati all'interno di una transazione non completata. Questo approccio può essere utile per evitare blocchi e aumentare le performance, ma in contesti come questo, dove la coerenza è fondamentale, può portare a gravi problemi, come dimostrato da questo caso.

Una volta individuata la causa del problema, è stato preparato uno script correttivo per l'eliminazione dei dati anomali, riportando il sistema in uno stato coerente. Questo episodio ha evidenziato l'importanza della corretta gestione delle letture in ambienti transazionali, e ha permesso di discutere internamente l'eventuale revisione dell'uso di WITH (NOLOCK) in contesti critici come i processi di housekeeping.

Un approfondimento sul WITH (NOLOCK), pro e contro

L'uso di WITH (NOLOCK) è una tecnica abbastanza comune nei sistemi con tanti accessi concorrenti. Serve per velocizzare le query, perché permette di leggere i dati senza aspettare

che siano rilasciati i lock dalle transazioni in corso. In pratica, si riescono a leggere i record di una tabella, anche se qualcuno sta contemporaneamente scrivendo nella stessa.

I vantaggi principali sono:

- Le query di lettura vengono eseguite più rapidamente, poiché non devono attendere il completamento di altre transazioni in corso.
- Si riduce il rischio di blocchi o deadlock.
- Migliora la capacità del sistema di gestire un alto numero di accessi concorrenti, aumentando la performance generale.

Però ci sono anche degli svantaggi importanti:

- Si possono leggere dati “sporchi”, cioè ancora non salvati ufficialmente, come è successo nel nostro caso.
- Se si verificano incoerenze è più difficile effettuare debugging

In generale, WITH (NOLOCK) può essere utile in certe situazioni, ma va usato con molta attenzione, soprattutto se i dati che si leggono servono per generare o aggiornare altri record. In casi critici, è meglio valutare alternative più sicure, per evitare di leggere dati in piena fase di scrittura.

Caso di bug: differenza di tipo tra colonne correlate

Un altro bug interessante su cui ho lavorato riguardava una discrepanza tra il tipo di dato e la *collation* utilizzata in due tabelle strettamente correlate del database: **tour** e **itinerariTour**. La tabella **tour** contiene le informazioni generali di un'attività terrestre, come titolo, codice, città e descrizione, mentre la tabella **itinerariTour** associa queste attività a una o più date specifiche, indicando il giorno in cui ogni tour è prenotabile.

Il problema si manifestava soprattutto a livello di visualizzazione: in alcune schermate il titolo dei tour appariva con caratteri strani, simboli errati o lettere accentate visualizzate male. Curiosamente, lo stesso dato poteva essere mostrato correttamente in un punto dell'applicazione e corrotto in un altro, rendendo difficile capire subito dove fosse il problema.

Dopo un'analisi più approfondita, ho scoperto che nella tabella **tour** i campi **codice** e **titolo** erano di tipo **VARCHAR**, mentre nella tabella **itinerariTour** gli stessi campi erano **NVARCHAR**. Oltre a questo, le due colonne **titolo**, avevano anche *collation* diverse, ovvero regole differenti per la gestione dei caratteri.

Questa differenza creava problemi soprattutto nelle query che coinvolgevano entrambe le tabelle, come **JOIN** o **WHERE** o. In pratica, anche se due valori apparivano identici a occhio nudo, venivano trattati in modo diverso a livello tecnico e potevano produrre risultati errati, errori di conversione implicita o visualizzazioni sbagliate.

Cos'è una collation?

La **collation** definisce le regole con cui il database confronta e ordina i caratteri. Ad esempio, stabilisce se "A" e "a" devono essere considerati uguali (case sensitivity), se "e" e "è" sono la stessa cosa (accent sensitivity) e in che ordine vanno ordinati i valori alfabeticamente.

Quando si confrontano due colonne con collation diverse, SQL Server può restituire un errore oppure forzare una conversione automatica che, in certi casi, altera i dati o restituisce risultati non coerenti.

Analisi e risoluzione

Una volta identificata la causa, ho risolto il problema uniformando entrambe le tabelle:

- Ho convertito i campi testuali coinvolti in **NVARCHAR**, per supportare correttamente anche i caratteri Unicode.
- Ho impostato la stessa collation su entrambe le tabelle, scegliendo una collation **CI_AI** (Case Insensitive, Accent Insensitive), per evitare problemi legati a maiuscole/minuscole o caratteri accentati.

Dopo questa modifica, i problemi di visualizzazione sono spariti e le query tra le due tabelle sono tornate stabili e coerenti. È stato un buon esempio di come dettagli apparentemente secondari, come il tipo di dato o la collation, possano avere un impatto concreto sull'esperienza utente e sulla correttezza dei dati.

Analisi e normalizzazione dell'architettura dati

Uno dei compiti più importanti che mi è stato assegnato una volta che mi sono familiarizzato con la struttura della base dati è stato fare un'analisi delle relazioni tra le tabelle che servivano a gestire gli itinerari delle navi e le tabelle che servivano a salvare gli itinerari dei tour terrestri. Infatti, come accennato nella sezione precedente, sono sorte diverse problematiche legate alla duplicazione delle informazioni, alla mancanza di normalizzazione e alla difficoltà di gestione coerente dei dati tra viaggi ed escursioni. In particolare, l'utilizzo del campo **idViaggio** all'interno della tabella **itinerariTour** ha generato una forte dipendenza tra ogni escursione e un singolo viaggio, impedendo il riutilizzo flessibile degli itinerari su più date o viaggi simili. Questo ha reso necessario avviare un processo di analisi e proposta di normalizzazione dell'architettura dati, con l'obiettivo di migliorarne l'efficienza, la coerenza e la scalabilità.

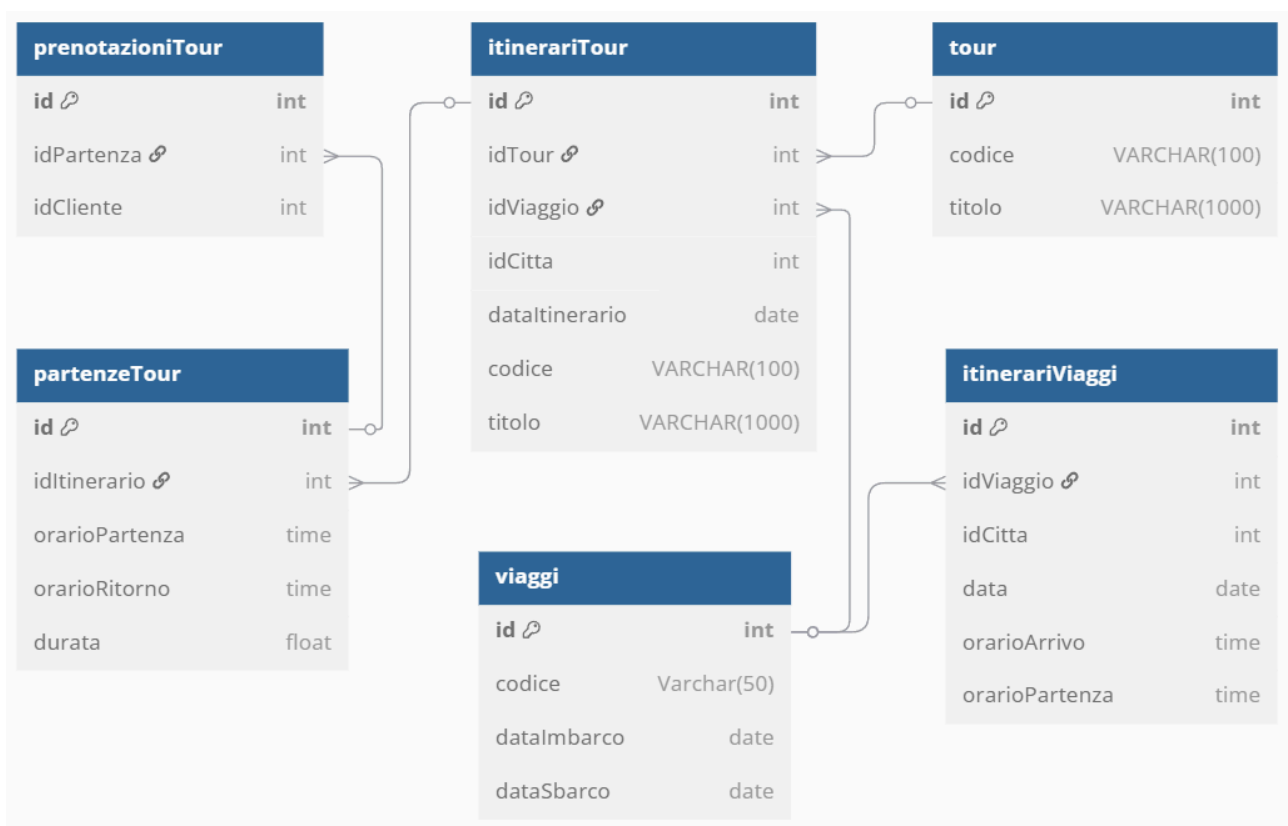
La soluzione che illustrerò nelle prossime sezioni non rappresenta la normalizzazione ideale dal punto di vista teorico, ma è un compromesso efficace che migliora sensibilmente la

situazione attuale e soprattutto rispetta tutti i requisiti funzionali richiesti dal sistema e dal cliente.

Analisi architettura attuale

La struttura del database attualmente in uso presenta una separazione tra i tour e i viaggi, ma mantiene un forte legame tra gli itinerari (**itinerariTour**) e i singoli viaggi (**viaggi**), attraverso il campo **idViaggio**. Ogni itinerario è dunque associato a un solo viaggio, anche nel caso in cui l'escursione sia identica per altri viaggi o ricorrente su più date. Questo vincolo limita la riusabilità dei dati e genera ridondanza, poiché la stessa escursione viene duplicata ogni volta che compare in un viaggio differente.

Di seguito uno schema semplificato delle principali tabelle coinvolte:



In questo schema si evidenzia come la tabella **itinerariTour** sia strutturalmente dipendente da **viaggi**, nonostante le escursioni siano spesso comuni a più viaggi con identiche caratteristiche. Questa relazione rigida rappresenta uno dei punti principali su cui si è concentrata l'analisi, in vista di una proposta di ristrutturazione più flessibile e normalizzata.

Problematiche di un database non correttamente normalizzato

Durante l'analisi della struttura del database, sono emersi diversi problemi causati dalla mancanza di una buona normalizzazione, soprattutto nella gestione degli itinerari e delle

escursioni. In particolare, abbiamo notato che il sistema attuale crea molti duplicati e rende complicato mantenere le informazioni sincronizzate.

Esempio pratico di duplicazione

Immaginiamo un'escursione come "Visita alla Sagrada Familia", proposta in più viaggi nella stessa città, lo stesso giorno. Anche se si tratta della stessa attività, il sistema attuale genera una riga distinta nella tabella **itinerariTour** per ogni viaggio che include quel tour. Ad esempio:

id	idTour	idViaggio	idCitta	dataItinerario	codice	titolo
1	101	1001	1	2025-07-03	BARC001	Visita alla Sagrada Familia
2	101	1002	1	2025-07-03	BARC001	Visita alla Sagrada Familia

Anche se la data, la città e il tour sono identici, vengono creati due itinerari separati solo perché i viaggi sono diversi. Questo comportamento genera problemi sia in fase di lettura che di modifica: ad esempio, se bisogna aggiornare l'orario o cambiare il titolo del tour, diventa complicato farlo con delle query SQL, perché bisogna identificare tutti i duplicati collegati. Il rischio di fare errori è alto, soprattutto quando ci sono tanti viaggi e tante righe simili. Ed è proprio quello che succedeva: capitava spesso che alcune righe rimanessero non aggiornate o che venissero modificate in modo non corretto, creando incoerenze nei dati.

Per cercare di tenere tutto sotto controllo, si ricorreva spesso a operazioni di housekeeping: controlli manuali o script dedicati per sistemare i dati, allineare titoli e codici, e rimuovere eventuali duplicati non più validi. Ma anche questa soluzione non è sostenibile nel lungo periodo, perché diventa sempre più difficile da gestire man mano che aumentano i tour, le date e i viaggi disponibili.

Va detto che questa scelta architetturale, cioè di duplicare l'itinerario per ogni viaggio, era stata fatta con uno scopo preciso: gestire in modo separato la prenotabilità delle escursioni a seconda del tipo di passeggeri presenti quel giorno. In questo modo, era possibile avere un itinerario visibile solo per i clienti che si imbarcavano, o solo per chi sbarcava, o solo per chi era già a bordo durante un cambio viaggio. Tuttavia, questa soluzione ha portato più problemi che vantaggi, ed è proprio su questo punto che si è deciso di intervenire con un nuovo modello.

Nuovo modello proposto

Dopo l'analisi delle criticità emerse nella struttura attuale, si è deciso di proporre un nuovo modello dati che punta a semplificare la gestione degli itinerari e ridurre le duplicazioni. L'obiettivo principale è eliminare il collegamento diretto tra un itinerario e un singolo viaggio, rendendo la struttura più flessibile e meno soggetta a errori.

Rimozione del collegamento con idViaggio

Nel vecchio modello, ogni itinerario era legato direttamente a un viaggio specifico tramite il campo **idViaggio**. Questo portava alla creazione di più righe identiche nella tabella **itinerariTour** ogni volta che la stessa escursione veniva proposta a più viaggi nella stessa città e nella stessa data. Con il nuovo modello, questo campo viene rimosso.

Ora un itinerario viene identificato univocamente dalla combinazione di:

- idTour
- dataItinerario
- idCittà

Questo consente di creare un solo itinerario per ogni data e città, indipendentemente dal numero di viaggi coinvolti, evitando così la duplicazione inutile delle righe.

Gestione della disponibilità con flag dedicati

Nel vecchio modello, la duplicazione degli itinerari serviva a distinguere per quali viaggi un'escursione fosse prenotabile: per chi si imbarcava, per chi sbarcava o per chi proseguiva su viaggi lunghi. Ora questa logica viene gestita tramite tre nuovi campi booleani nella tabella **itinerariTour**:

- **disponibilitaImbarco**: Indica se l'itinerario è disponibile per i passeggeri che iniziano il viaggio in quella data.
- **disponibilitaSbarco**: Specifica se è prenotabile da chi termina il viaggio in quella data.
- **disponibilitaViaggiLunghi**: Serve a gestire i passeggeri già a bordo per viaggi combinati o crociere lunghe, che non sbarcano né si imbarcano in quella data.

Questa nuova modalità permette di controllare con precisione la visibilità dell'itinerario, mantenendo una sola riga per ogni combinazione reale di tour, città e data.

Esempio pratico

Con il vecchio schema, se tre viaggi diversi erano a Napoli il 15 agosto e tutti offrivano lo stesso tour "Visita a Pompei", venivano generate tre righe uguali, una per ogni **idViaggio**.

Con il nuovo schema, c'è una sola riga in **itinerariTour**, e i flag **disponibilitaImbarco**, **disponibilitaSbarco** e **disponibilitaViaggiLunghi** vengono impostati in base alla tipologia di viaggi presenti in quella data.

Ad esempio:

- Se l'escursione è per chi si imbarca: **disponibilitaImbarco** = TRUE
- Se è per chi sbarca: **disponibilitaSbarco** = TRUE
- Se è per chi resta a bordo: **disponibilitaViaggiLunghi** = TRUE

Questo nuovo approccio risolve il problema dei duplicati, riduce la possibilità di errori nei dati e rende molto più semplice la gestione, l'aggiornamento e l'evoluzione del sistema. Inoltre, garantisce maggiore coerenza e flessibilità, senza compromettere le funzionalità legate alla prenotazione dei tour.

Impatto e rischi delle modifiche sui processi attuali

L'introduzione del nuovo modello dati porta sicuramente una serie di vantaggi in termini di semplificazione, manutenzione e coerenza, ma comporta anche degli impatti rilevanti sui processi esistenti e alcuni rischi da considerare attentamente, soprattutto nella fase di transizione.

Impatto sulle stored procedure e sulla logica esistente

Il cambiamento più evidente è che, eliminando il campo **idViaggio** dalla tabella **itinerariTour**, tutte le stored procedure e le query che si basavano su quel campo per estrarre gli itinerari disponibili dovranno essere riviste. In particolare:

- Bisogna modificare tutte le SP che estraggono i tour prenotabili da un viaggio specifico, basandosi ora sulla combinazione di data, città, nave e sui tre flag (**disponibilitaImbarco**, **disponibilitaSbarco**, **disponibilitaViaggiLunghi**).
- Le condizioni di JOIN e di filtro cambieranno radicalmente, e bisogna assicurarsi che la nuova logica restituisca risultati coerenti con il comportamento atteso.
- Ogni modifica va testata in ambienti di staging, per verificare che non ci siano effetti collaterali imprevisti sulle funzionalità già in uso.

Per avere un'idea approssimativa delle tempistiche e dell'ammontare di lavoro necessario, ho creato un excel apposito con il calcolo dell'effort per aggiornare tutte le Stored Procedures coinvolte. Di seguito uno screenshot di esempio tratto da quel file:

Database name	Schema name	Object name	Type	Effort
			SQL_STORED_PROCEDURE	Alto
			SQL_STORED_PROCEDURE	Alto
			SQL_STORED_PROCEDURE	Nessuno
			SQL_STORED_PROCEDURE	Basso
			SQL_STORED_PROCEDURE	Nessuno
			SQL_STORED_PROCEDURE	Nessuno
			SQL_STORED_PROCEDURE	Medio
			SQL_STORED_PROCEDURE	Basso
			SQL_STORED_PROCEDURE	Medio
			VIEW	Medio

Essendo una stima approssimativa, ho deciso di suddividere le stored procedure in quattro livelli di complessità:

- Nessuno: la SP non necessita modifiche.
- Basso: la modifica è semplice e può essere completata nel giro di qualche ora.
- Medio: richiede un intervento più strutturato, che può occupare mezza giornata o una giornata intera di lavoro.
- Alto: usato per le Stored Procedure più delicate, con logica complessa o impatti trasversali. In questo caso ho stimato un effort minimo di 1-2 giorni, inclusi test accurati e validazione dei risultati.

Questa classificazione è pensata per aiutare nella pianificazione degli interventi e nella gestione delle priorità, in modo da affrontare prima le modifiche a basso impatto e concentrarsi con il giusto tempo e risorse su quelle più critiche.

Rischi principali

Alcuni dei rischi che abbiamo individuato durante la fase di analisi sono:

- Modifiche errate dei flag: Se i flag non vengono valorizzati correttamente, si rischia di rendere un tour visibile a chi non dovrebbe vederlo, oppure di nascondere a chi invece dovrebbe poterlo prenotare.
- Gestione dei giorni di cambio viaggio: In caso di modifiche al calendario dei viaggi (es. spostamento di un giorno di imbarco o sbarco), i flag sui tour esistenti potrebbero non essere più coerenti. Servirà prevedere dei controlli o delle logiche automatiche per ricalcolarli quando necessario.
- Perdita della granularità "per viaggio": Con la rimozione del campo **idViaggio**, non sarà più possibile avere un itinerario completamente personalizzato per un singolo viaggio. Questo è un compromesso da tenere in considerazione.
- Aggiornamento massivo dei dati: Il passaggio al nuovo modello richiederà uno script di migrazione per raggruppare correttamente gli itinerari duplicati e impostare i flag sulla base della logica attuale. Un errore in questa fase può compromettere la qualità del dato e generare comportamenti imprevisti in produzione.

Soluzioni applicate

Anche se il nuovo modello non è stato ancora implementato in ambiente produttivo, è già stata svolta un'analisi dettagliata sulle soluzioni tecniche da adottare per rendere la transizione il più sicura ed efficiente possibile. L'obiettivo è applicare la nuova struttura senza downtime, minimizzando l'impatto sui sistemi in uso.

Strategia di migrazione senza interruzioni

La soluzione ipotizzata prevede uno script di aggiornamento dati che possa essere eseguito in parallelo all'ambiente live. In questo modo, si evita il blocco delle operazioni e si garantisce

la continuità dei servizi durante tutta la fase di transizione. La strategia è suddivisa in più step:

1. Verranno aggiunti alla tabella **itinerariTour** i tre flag:
 - a. **disponibilitaImbarco**
 - b. **disponibilitaSbarco**
 - c. **disponibilitaViaggiLunghi**
2. Sarà creato uno script per analizzare gli itinerari esistenti (basandosi su **dataItinerario**, **idCitta**, **idTour** e **idViaggio**) e calcolare i flag corretti.
3. In parallelo, si comincerà a raggruppare logicamente gli itinerari duplicati per simulare la futura chiave composta. Le nuove query potranno essere testate con questi gruppi per verificarne il corretto funzionamento.
4. Verranno modificate tutte le SP che attualmente usano **idViaggio** per estrarre gli itinerari. Le nuove versioni utilizzeranno i flag appena aggiunti. Il rilascio sarà graduale, con ambienti di test dedicati.
5. Prima del rilascio definitivo, verranno eseguiti controlli automatici e manuali per confrontare i risultati delle vecchie e nuove logiche e verificare che non ci siano anomalie nei tour visualizzati.
6. Solo dopo un periodo di verifica si procederà con l'attivazione della nuova logica in produzione. I dati duplicati verranno mantenuti temporaneamente per motivi di tracciabilità, ma non più utilizzati dalle query.

Il passaggio al nuovo modello rappresenta un cambiamento strutturale importante e delicato. Tuttavia, la pianificazione dettagliata e l'approccio step-by-step sono pensati per garantire una transizione fluida, sicura e con interruzioni minime del servizio.

Conclusioni

L'esperienza di stage presso Widesolutions ha rappresentato per me un momento fondamentale di crescita sia professionale che personale. Fin dall'inizio ho avuto l'opportunità di lavorare in un contesto stimolante, dove ho potuto mettere in pratica le competenze acquisite durante gli studi e, allo stesso tempo, apprendere nuove conoscenze direttamente sul campo.

Durante lo stage ho maturato competenze tecniche rilevanti soprattutto nell'ambito dei database e della loro gestione. L'esperienza mi ha permesso di approfondire l'uso del linguaggio SQL in un contesto reale, comprendendo meglio le logiche che regolano l'interazione tra applicazioni e basi di dati. Ho imparato a scrivere query efficaci, a leggere e interpretare strutture complesse e a intervenire per ottimizzare le prestazioni.

Oltre all'aspetto puramente linguistico, ho avuto modo di conoscere strumenti professionali utilizzati per la gestione e il monitoraggio dei database, che mi hanno aiutato a capire come individuare eventuali criticità e come gestire modifiche in modo ordinato e sicuro. Questa parte dello stage è stata particolarmente formativa perché mi ha fornito una base solida su cui costruire ulteriori competenze nel campo della programmazione e dell'analisi dei dati.

Sul fronte dello sviluppo back-end, ho avuto modo di utilizzare Visual Studio e .NET per lavorare su codice C#. Questo mi ha permesso di affinare le mie competenze nella programmazione, con particolare attenzione alla logica applicativa e alle esigenze funzionali del cliente.

Dal punto di vista organizzativo, ho imparato a gestire le attività in modo più strutturato ovvero definire le priorità, stimare correttamente i tempi, documentare il lavoro svolto e confrontarmi in modo efficace con i referenti tecnici del progetto. Questo mi ha reso più autonomo nella gestione dei compiti e più consapevole delle dinamiche del lavoro in team.

In conclusione, considero questo stage un'esperienza estremamente positiva, che mi ha permesso di consolidare competenze tecniche, sviluppare capacità operative e crescere come professionista. Mi ha dato gli strumenti per affrontare con maggiore sicurezza le sfide future e ha rafforzato la mia motivazione a proseguire con questo percorso lavorativo.

Bibliografia/Sitografia

Strumenti utilizzati per la creazione di diagrammi e schemi:

- dbdiagram.io: Utilizzato per creare diagrammi E-R e visualizzare relazioni tra tabelle database.
- diagrams.net: Usato per la realizzazione di schemi concettuali e diagrammi generici.
- mermaidchart.com: Sfruttato per creare diagrammi con sintassi testuale tramite Mermaid.js.

Documentazione tecnica (Microsoft Learn):

- [COLLATE \(Transact-SQL\) - SQL Server | Microsoft Learn](#). Documentazione ufficiale sull'utilizzo e la gestione della collation nelle query SQL.
- [Hint di tabella \(Transact-SQL\) - SQL Server | Microsoft Learn](#) Spiegazione dei suggerimenti (hints) utilizzabili nelle query per influenzare il comportamento dell'ottimizzatore SQL Server.
- [ROW_NUMBER \(Transact-SQL\) - SQL Server | Microsoft Learn](#) Guida all'utilizzo della funzione ROW_NUMBER() per assegnare numeri di riga a risultati di query.